

A photograph of two men in a modern office setting, viewed from the side. They are seated at a desk, looking at several computer monitors. The man on the left has a full beard and is wearing a dark jacket over a plaid shirt. The man on the right is wearing a plaid shirt. The monitors display lines of code in a dark-themed editor. A large window in the background shows a cityscape with buildings. A white text box is overlaid on the image, containing the title.

Objekter med tilstand og adfærd

Program:

- Objekter med tilstand og adfærd
- Øvelser
- Opsummering

Vi har set klasser anvendt som

- **navnerum** til at gruppere relaterede metoder, fx `DistanceConverter`
- **egne datatyper**, fx `Contact`

Vi har set objekter, fx **Contact** anvendt som

- databærere med felter, fx name og email
- *uforanderlige* (immutable) værdier

I dag skal vi

- se på objekter, der både har tilstand *og* adfærd
- se på objekter, der kan ændre tilstand (er mutable)

Som eksempel skal vi lave en simpel bankapplikation

Spørgsmål: Hvad data har en bankkonto?

DEMO: En simpel bankkonto med et kontonummer og en saldo

Jeg vil gerne kunne indsætte penge på kontoen

```
BankAccount account = new BankAccount(337898);  
account.balance = 100;
```

Jeg kunne også

```
BankAccount account = new BankAccount(337898);  
account.balance = 10000000000000000;
```



Kan I se problemet?

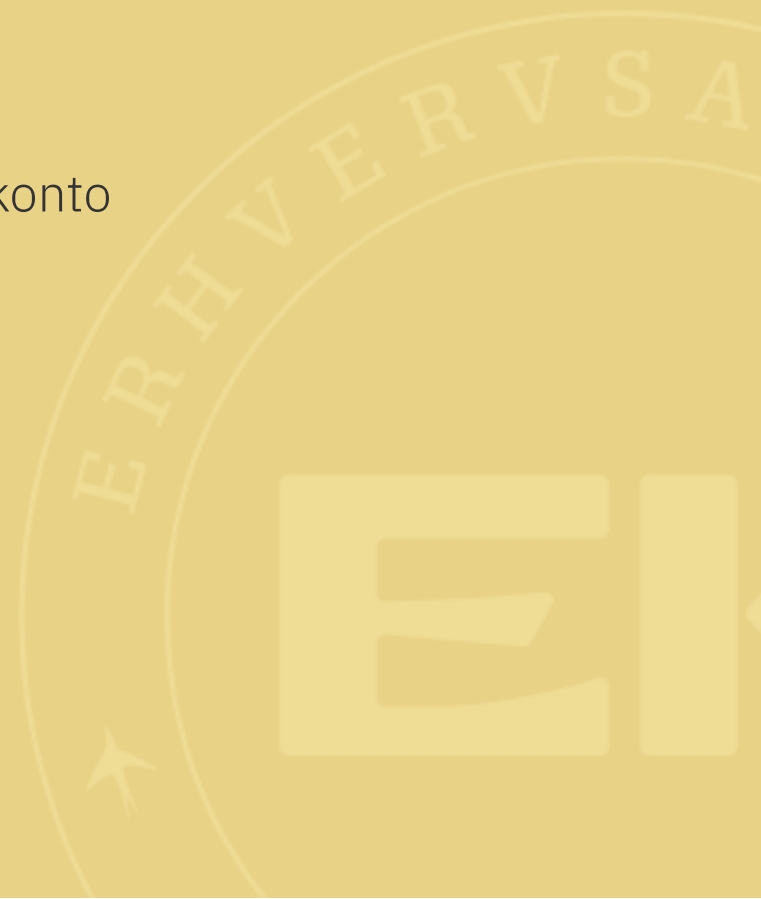
Vi må have regler for hvordan **balance** kan ændres

Lad os tilføje adfærd til **BankAccount**-klassen:

- en `deposit`-metode til at indsætte penge
- en `withdraw`-metode til at hæve penge

Hvilke regler skal der gælde for at hæve penge?

DEMO: En simpel bankkonto



Spørgsmål: Hvorfor er der ingen `static` i `deposit` og `withdraw`?

Fordi de skal virke på en bestemt konto, dvs. et bestemt objekt.

Derfor skal de være instansmetoder, dvs. uden **static**.

```
BankAccount account = new BankAccount(337898);  
account.deposit(100);  
account.withdraw(50);
```

```
BankAccount.withdraw(50); // fejl  
BankAccount.deposit(100); // fejl
```



Objekter som argumenter og returværdier

Vi har mødt mange metoder, der tager **String** som argument og/eller returværdi, fx

```
public static String toUpperCase(String s) {  
    // ...  
}
```

```
public static boolean transfer(BankAccount from, BankAccount to, double amount) {  
    // ...  
}
```

Bare tænk: **String** erstattet med **BankAccount** - de er begge reference-typer og objekter. Det samme gælder arrays af objekter.

A photograph of three people in a meeting. On the left, a man in a white t-shirt looks intently at a laptop. In the center, a woman with blonde hair, wearing a dark blazer over a white top, smiles at the camera. On the right, another person is partially visible, holding a pen. The laptop screen shows a web application with various charts and data. A white text box is overlaid in the center of the image.

Arrays af objekter

Kan I huske String-arrays?

```
String[] playlist = new String[3];  
playlist[0] = "Lukas Graham - 7 Years";  
playlist[1] = "Adele - Hello";  
playlist[2] = "MØ - Final Song";  
  
System.out.println(playlist[2]); // MØ - Final Song
```

Prøv med **BankAccount**-objekter

```
BankAccount[] accounts = new BankAccount[3];
accounts[0] = new BankAccount(337898);
accounts[1] = new BankAccount(337899);
accounts[2] = new BankAccount(337900);

accounts[0].deposit(100);
System.out.println(accounts[0].balance); // 100.0
```



```
for (String song : playlist) {  
    System.out.println(song);  
}
```

```
for (BankAccount account : accounts) {  
    System.out.println("Saldo på konto " + account.accountNumber + ": " + account.balance);  
}
```

A photograph of three people in a meeting. On the left, a man in a white t-shirt looks intently at a laptop. In the center, a woman with blonde hair, wearing a dark blazer over a white top and a pearl necklace, looks towards the camera with a slight smile. On the right, another person is partially visible, holding a pen. The laptop screen displays a colorful dashboard with various charts and graphs. A white cup and a blue folder are on the table.

Hvad er null

Kan I huske den med at en variabel er en navngivning af en plads i hukommelsen?

```
int age;  
System.out.println(age); // Fejl: variable age might not have been initialized  
  
String name;  
System.out.println(name); // Fejl: variable name might not have been initialized
```

Vi kan ikke kompilere, fordi **age** og **name** ikke er initialiseret.

Men med **null** kan vi have en variabel, der *ikke* peger på noget objekt, men er initialiseret

```
String name = null;  
System.out.println(name); // null
```

dvs. **name** peger ikke på noget **String**-objekt, men er stadig initialiseret.

Hvordan? Fordi **null** er en reference-værdi, der ikke peger på noget objekt.

På engelsk bruger vi ordet "pointer" for den pil, der peger på et objekt i hukommelsen. **null** er en "null pointer", dvs. en pil, der ikke peger på noget som helst.

NullPointerException (NPE) er en meget almindelig fejl i Java-programmering.

Det sker når vi prøver at bruge en variabel, der er **null**, som om den pegede på et objekt, fx

```
String name = null;  
System.out.println(name.length()); // NullPointerException
```

A.k.a. vi troede name var en **String**, som har en **length()** - men det havde den jo ikke, da den var **null**.

Alligevel er det meget udbredt at bruge **null** til at indikere "ingen værdi", fx i en metode, der skal returnere et objekt

```
public static BankAccount findAccount(int accountNumber) {  
    if ( /* account not found */ ) {  
        return null; // ingen konto fundet  
    }  
    return new BankAccount(accountNumber); // konto fundet  
}
```

og det er så vores ansvar at tjekke for **null**, når vi kalder metoden

```
BankAccount account = findAccount(337898);  
if (account != null) {  
    System.out.println("Konto fundet: " + account.accountNumber);  
} else {  
    System.out.println("Konto ikke fundet");  
}
```

Så i Java skal man ofte holde styr på om en variabel er `null` eller peger på et objekt

The background is a blurred photograph of a crowd of people. On the right side, the face of a person with red hair and green eyes is partially visible, looking towards the camera. The text 'Garbage collector (GC)' is overlaid on a white rectangular box in the center-left of the image.

Garbage collector (GC)

Husker i stack'en og heap'en?

- primitive typer (int, double, boolean, ...) ligger i stack'en (lille)
- reference-typer (objekter) ligger i heap'en (stor)

Primitive typer slettes automatisk når de går **ud af scope**, fx

```
public static void foo() {  
    int x = 42;  
    // x eksisterer her  
}  
// x eksisterer ikke her
```

```
if (true) {  
    double y = 3.14;  
    // y eksisterer her  
}  
else {  
    // y eksisterer ikke her  
}
```

For reference-typer (objekter) er det anderledes.

Referencen til et objekt ligger i stack'en og slettes når den går ud af scope

```
public static void foo() {  
    BankAccount account = new BankAccount(337898);  
    // account eksisterer her og peger på et BankAccount-objekt i heap'en  
}  
// account-referencen eksisterer ikke her
```

... MEN objektet ligger stadig i heap'en og fylder, selvom vi ikke kan komme til den.

En anden måde at miste referencen til et objekt er at sætte variabelen til **null**

```
BankAccount account = new BankAccount(337898);  
account = null;  
// nu peger account-referencen ikke længere på BankAccount-objektet
```

Eller sætte den til at pege på et andet objekt

```
BankAccount account = new BankAccount(987654);  
account = new BankAccount(123456);  
// nu peger account-referencen på et andet BankAccount-objekt
```

Nu kan vi ikke længere komme til **BankAccount** en 987654.

Fylder vi så heap'en op med ubrugte objekter? (Tænk: rumskrot)

Nej, heldigvis har Java en flittig rengøringshjælp, der hedder **garbage collector** en.

Garbage collector'en

- er en process, der kører i baggrunden i Java Virtual Machine (JVM)
- opdager objekter, der ikke længere er refereret af nogen variabel
- frigør heap-hukommelse ved at slette de ubrugte objekter

Har alle programmeringssprog en garbage collector?

Nej, fx C og C++ har ikke en garbage collector. I de sprog er det programmørens ansvar at frigøre hukommelse, når den ikke længere er nødvendig.

Nævn tre ting du tager med fra i dag?